

Preservation of Chaotic Dynamics in Minimally Structured Neural Networks

Matty Boles, Frank Kwasniok

September 2026

1 Abstract

The aim of this project is to investigate the ability of minimally structured multi-layer perceptrons to learn the dynamics of a chaotic dynamical system from prediction error alone, and to identify and diagnose shortcomings and failure modes. Using the canonical test bed of Lorenz-63, we investigate the choice of optimiser, network width and activation function. We employ Lyapunov spectra and singular value decomposition of the discretised flow map, to evaluate a model's success. We find that a curvature-aware optimiser is more effective at converging at a low-MSE minimum, consistent with the ill-conditioned loss landscape with highly coupled parameters. Analysis of activation functions highlights a structural limitation of ReLU caused by the piecewise-constant derivative. We identify a failure mode of tanh, in which singular value decomposition shows sharp increases in contraction, unrepresentative of the true system. Softplus avoids this failure mode, and captures the dynamics more consistently, especially in narrow systems. By testing a range of widths on Lorenz-63, we find increased width provides more consistently accurate dynamics. However, investigating the generalisation to Lorenz-96 instead shows a minimum viable width, for which higher widths show no additional benefit. These results demonstrate that small multi-layer perceptrons can reproduce the important properties of chaotic systems, but their success is highly variable and performance cannot be established by prediction error alone.

2 Introduction

Data-driven prediction of chaotic systems using machine learning methods is an ever-changing research field, important for improving accuracy of forecasting in areas such as weather prediction. Neural networks offer significant computational advantages over classical methods for chaotic systems, but their utility depends on preserving the chaotic properties of the true system. The mechanisms determining whether, and how, a network preserves chaos, remains insufficiently understood. This is the objective of this project, using Lyapunov spectra [1] and singular value decomposition (SVD) [2], introduced shortly, to compare a variety of neural networks on the Lorenz-63 (L63) [3] equations.

2.1 Chaotic Dynamical Systems

Chaotic systems are dynamical systems in which infinitesimal perturbations of initial conditions cause trajectories to separate exponentially. Although they are deterministic, they exhibit aperiodic behaviour [4], [5]. Due to tiny errors compounding, accurate long-term prediction of chaotic systems, past a finite time horizon, is impossible. Even floating point errors can cause the prediction to exponentially separate from the true solution. For this reason, we focus the project on learning the long-term statistical and dynamical properties of a system, which are invariant to initial conditions.

2.2 Literature Review and Research Gap

Neural networks of many architectures have been studied extensively as surrogates of chaotic dynamical systems.

In [6], Vlachas et al use a variant of a long-short term memory (LSTM) [7] model, to demonstrate the ability of a recurrent neural network (RNN) [8] to provide a slow deterioration of root mean squared error over long trajectories. Although RNNs are shown to be effective at reproducing chaotic dynamical systems, due to their use of an internal hidden state to make predictions, this architecture doesn't preserve the explicit Markovian structure of the system.

In [9], Shahia et al compare RNNs to reservoir computing (RC) [10], [11] solutions on various chaotic dynamical systems including L63. RC is an implementation of an RNN in which a hidden state of interconnected nodes are evolved over time, while only the read-out layer is trained. Shahia et al successfully show that while RNNs can cause dynamical systems to collapse to a point, destroying the chaotic nature of the equations, the RC networks maintain accurate predictions throughout a long trajectory. Similar results have been shown in [12] by Gauthier et al, on the L63 equations, and in [13] by Pathak et al, in which the Kuramoto-Sivashinsky equations are used as a test bed, and the first Lyapunov exponent is successfully retrieved. RC solutions are often the preferred architecture for high-performance forecasting of dynamical systems. As well as the bias intrinsic to RNNs, RC introduce an additional inductive bias through its fixed reservoir dynamics. Parameters such as the spectral radius of the reservoir can be used to influence the dynamical regime, and as such, the dynamics are largely predetermined, rather than learnt.

Each of the above papers uses 1-step prediction error as a metric for evaluating a model for reproducing the dynamical system on which it is trained. However, a low prediction error alone is insufficient to evaluate the dynamics of a system. Indeed, in [14], Margazoglou and Magri focus their work directly on the stability analysis of Echo-State Networks, a form of RC, and derive the Jacobian. They use Lyapunov exponents and covariant Lyapunov vectors to evaluate the success of a model. These are more informative metrics. This approach provides a strong motivation for this report.

In [15], Park et al evaluate the use of Neural ODE’s [16] for dynamical systems, again with L63 as a test bed, utilising a loss function which incorporates the Jacobian of the system. They importantly highlight that evaluating a model on 1-step error alone cannot be used to determine whether the dynamics of a system are successfully retrieved. As such, they analyse Lyapunov exponents and statistical measures to evaluate the models. Neural ODE’s, as expected, are very effective at recreating the dynamics of a system, as the dynamics are directly incorporated into the loss function. This is a much smaller bias than that exhibited by other architectures, as it is true to the Markovian nature of the systems, and only assumes the system can be represented as a continuous time dynamical system. Indeed, a study on Neural ODEs would be an appropriate follow-up to this report.

However, the direct emphasis on the dynamics gives rise to the overarching question of this paper. We, instead, focus on a simple multi-layer perceptron (MLP), in which only the 1-step prediction error is used to optimise the model, and evaluate the success of a model to learn the dynamics of a system without directly incorporating those dynamics into the training process. MLPs avoid several of the possible biases introduced by the above architectures, providing a minimally structured setting. However, in practise, MLPs are rarely used due to the better 1-step error of the architectures discussed above. We aim to understand how well the minimally biased architecture can learn the underlying dynamics of the system from prediction error alone, and to understand specific limitations that arise. The limitations identified provide justification for introducing more complex architectures.

Indeed, MLPs have been implemented and studied for chaotic dynamical systems, and specifically L63. In [17], Balogh et al show that for more complex models, MLPs can struggle to find chaotic, bounded solutions by investigating an extended L63 scheme, and show that the classical L63 is not complex enough to exhibit these unstable trajectories. In [18], Li and Ravela effectively show the capabilities of a tiny, 4-neuron MLP to preserve the first Lyapunov exponent of the L63 equations, and provide a geometric reasoning as to how such a simple architecture is able to capture the long term dynamics of a chaotic system. The work of Li and Ravela provides the primary inspiration for this report. The demonstration that an incredibly small MLP can reproduce the first Lyapunov exponent raises an important question: to what extent has the minimally structured network actually learnt the underlying dynamics?

An accurate Lyapunov spectrum does not necessarily imply that a model has reproduced the local dynamics of the system. 2 trained models may exhibit long-term chaotic characteristics, while the local dynamics differ substantially. We therefore investigate the SVD of the discretised flow map.

To the best of our knowledge, the consistency of models to be able to capture the long term dynamics, and instantaneous geometry of the system has not been systematically investigated. We comparatively evaluate several activation functions and widths to see the influence they have on the ability of an architecture to consistently capture the dynamics of the L63 equations.

Finally, we test these findings on the Lorenz-96 (L96) equations, to see if they are generalisable.

These findings provide insight into what the minimally biased architecture can achieve, and diagnose key shortcomings mechanistically, to justify the use of more biased models in practise.

As we aim for the minimal viable network, we also focus our work to just 1 hidden layer, with the minimum viable width.

2.3 Analysis techniques

We use 2 general measures of a successful model:

The Lyapunov spectrum [1] of a system describes the rate of exponential separation in orthogonal directions. The first, most positive value, the Lyapunov exponent, λ_1 , dictates the overall separation of the system:

$$||x'(t)|| \sim e^{\lambda_1 t} ||x'(0)||,$$

where $x'(t)$ is the perturbation at time t . Hence, by rearranging, we find λ_1 :

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{||x'(t)||}{||x'(0)||}$$

To make this calculation easier, we define the tangent propagator, $M(t, t_0)$, which describes the evolution of the perturbation from time t_0 to t :

$$x'(t) = M(t, t_0)x'(t_0)$$

As the choice of initial time is arbitrary, we set $t_0 = 0$. Then, by substituting and rearranging, we obtain:

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{||M(t, 0)x'(0)||}{||x'(0)||}$$

By assuming perturbations are small, we can see they show linearised dynamics:

$$\dot{x}'(t) = J(t)x'(t)$$

Here, $J(t) = F'(x)|_{x=x(t)}$. Then, we find M satisfies:

$$\dot{M}(t, 0) = J(t)M(t, 0), \quad M(0, 0) = I,$$

which has the formal solution,

$$M(t, 0) = \mathcal{T} \exp \left(\int_0^t J(s) ds \right),$$

where, $\mathcal{T}$ is the time-ordering operator. A more in-depth derivation is provided in the Appendix A.

The Lyapunov exponent defines the overall chaos of a system:

- $\lambda_1 < 0$ is stable.
- $\lambda_1 = 0$ is neutral.
- $\lambda_1 > 0$ is chaotic.

More generally, the Lyapunov spectrum describes this in orthogonal directions. In this paper, we employ the Benittin [19] method of finding the Lyapunov spectra. This method evolves a set of tangent-space vectors according to the Jacobian. Every k time steps, these vectors are QR-decomposed [20]:

$$J = QR$$

Here, Q is orthonormal and R is upper triangular. Our orthonormal matrix Q becomes our tangent-space vectors. and the diagonal values of R are accumulated. Finally, we have:

$$\lambda_i = \frac{1}{n\Delta t k} \sum_{j=1}^n \ln |R_{i,i}^{(j)}|,$$

where n is the number of QR-decompositions and Δt is the time step. A successful model would have a Lyapunov spectrum, $\hat{\lambda}_i$, arbitrarily close to the true system, λ_i , for all i .

The singular value decomposition (SVD)[2] of the discretised flow map of the system describes local geometry by identifying orthogonal directions of instantaneous stretching and contraction. A singular value greater than 1 describes local stretch, exactly 1 describes a neutral direction and less than 1 describes local contraction. SVD represents a matrix, A , as :

$$A = U\Sigma V^T,$$

where:

- V contains the right singular vectors, which form a basis of the input space.
- Σ contains the singular values, σ_i , which are stretching factors.
- U contains the left singular vectors, which form a basis of the output space.

We focus our attention on σ_i , to ensure a well-trained model successfully recreates the local geometry throughout the system. Indeed, a model can have an accurate Lyapunov spectrum, as it is a long-term average, but fail to capture the local geometry throughout the space.

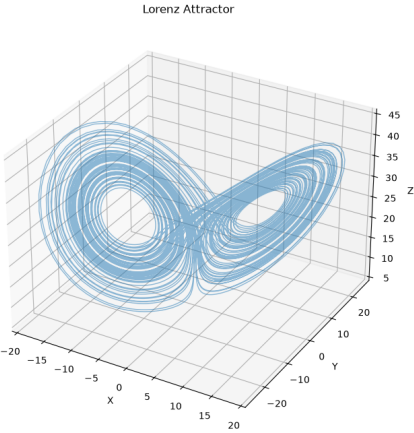
2.4 Lorenz-63

The Lorenz-63 (L63) equations portray a idealised system for convection cycles in the atmosphere. They were developed by Edward Lorenz in his 1963 paper [3], and have served as a test bed for non-linear dynamics since, due to the simplicity of the equations, and incredibly detailed, structured and sensitive resulting geometry.

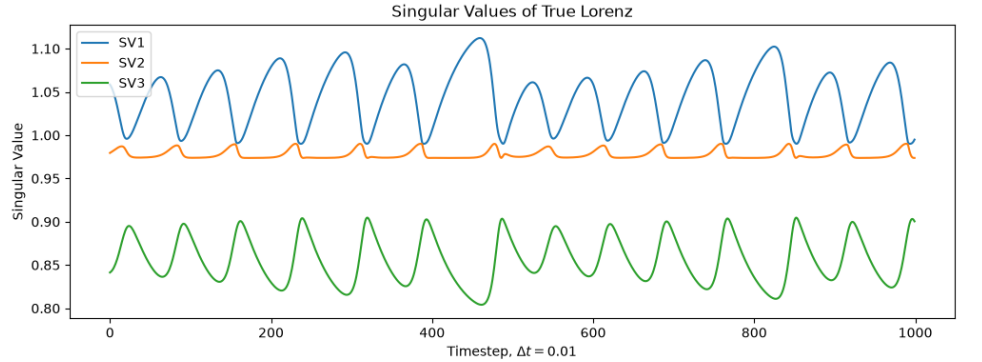
$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$

Here, x is proportional to the intensity of the convection motion, y is proportional to the temperature difference between ascending and descending currents, and z is proportional to the distortion of the vertical temperature profile from linearity. σ is the Prandtl number, ρ is the Rayleigh number and β is another, physical constant. We use the typical values, which ensure chaos, of $\sigma = 10, \beta = \frac{8}{3}, \rho = 28$. Further research comparing the ability of an architecture to learn the dynamical system when we change the chaotic mode by varying these parameters would be interesting.

These parameters of L63 give rise to the classical butterfly attractor. This is a 2-lobed shape, in which attracted trajectories orbit an unstable centre, until sufficient distance is reached, at which time the trajectory leaves the cycle of one centre and swaps to the other centre. This provides a system where it is difficult to predict the number of loops of a centre, before swapping. This gives rise to the chaos. The system has 1 saddle point at $(0,0,0)$, as well as the 2 unstable centres at $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$, which, with our parameters, are $(\pm6\sqrt{2}, \pm6\sqrt{2}, 27)$.



(a) The Lorenz attractor, with initial point $(2,4,9)$, iterated for 10000 time steps of length $\Delta t = 0.01$, using the RK4 method.



(b) The flow map singular values of the true Lorenz system, over 1000 time steps, with $\Delta t = 0.01$, using the RK4 method.

Figure 1: Lorenz Attractor and SVD of Flow Map

The Lyapunov spectrum of the true Lorenz system is :

- $\lambda_1 \simeq 0.906$
- $\lambda_2 \simeq 0$
- $\lambda_3 \simeq -14.57$

The SVD of the L63 flow map shows that the singular value describing stretching, σ_1 , oscillates between around 1 and 1.13, for each loop of a centre. The amplitude of the oscillations is proportional to the width of the loop, i.e. the amplitude of σ_1 grows with the radius of a loop, and after lobe switching, the amplitude drops back down. Similarly, σ_3 oscillates with amplitude proportional to the width of the current loop, within the range 0.77 and 0.91.

2.5 Methodology

All models were trained with pairs of inputs and output coordinates in (x,y,z). Care was taken to ensure all points are on the attractor by integrating over a 'transient period' which was then disregarded. Input and output pairs are found using the 4th order Runge-Kutta (RK4) [21] time stepping scheme, with $\Delta t = 0.01$, to discretise the solution.

3 Training Hyperparameters

We initially chose to implement the AdamW optimiser [22], a widely used optimiser in machine learning which employs first and second moment estimates of the gradients to determine parameter updates, as well as decoupled weight decay. These features allow it to optimise large neural networks efficiently and stably, while reducing overfitting, making it a conventional, go-to optimiser for many machine learning tasks. However, for our application, AdamW presented 2 major limitations.

The first is weight decay. Weight decay is regularization technique used by AdamW as default to reward simpler solutions by punishing weights of large magnitude. For parameter θ ,

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \gamma \frac{\hat{m}_t}{(\sqrt{\hat{v}_t} + \epsilon)} - \gamma \lambda \theta_{t-1} \end{aligned}$$

Here, t is the optimisation step, g is the gradient of the parameter with respect to the loss function, γ is the learning rate, and λ scales the weight decay.

As the L63 system is deterministic and contains no noise, the model can (theoretically) converge to an exact minimum. However, the weight decay continually pulls the parameters towards 0, even when the parameters have reached an optimum. This regularization is helpful in large, noisy datasets, where it can help to prevent overfitting, however it harms dynamical systems, in which accurate representation of the governing equations is required. In these cases, even tiny perturbations in weights can have large repercussions on the dynamics of the system.

In particular, in the L63 system, there are areas of high contraction and expansion, caused by non-linearities. These require relatively large weights to portray, and so a model with smaller weights than required lacks the peaks and troughs in the singular values of the Jacobian, which results in a more 'typical trajectory', in which loops are of more consistent radius, and rare areas of the space are not visited, such as orbits of narrow radius of the unstable points.

The second limitation of AdamW is that the scale of weight updates are determined largely parameter-wise, with only indirect influence from other parameters. The advantages of parameter-wise optimisation are less appropriate when the number of parameters is small and they are highly coupled, such as in a dynamical system. In such systems, the optimisation landscape can be highly ill-conditioned, meaning that different parameters may require updates of substantially different magnitudes. AdamW doesn't explicitly account for cross-parameter curvature, and as such updates may be suboptimal. For example, a large update in one parameter direction may cause another parameter to move away from its optimum.

We can picture, as an example, the loss landscape as a valley. AdamW finds the valley well. However, once in the valley, there is a direction of high curvature and a direction of low curvature, making the terrain ill-conditioned. AdamW has no explicit context of the local terrain, and performs parameter-wise rescaling, removing away the informative differences in gradient magnitude. This produces ineffective trajectories across the narrow directions of the valley, rather than following the direction of overall descent, which is a common problem for optimisers which do not use local curvature information.

By calculating the Hessian of loss with respect to parameters when training plateaus, we can show evidence of this failure mode. We ran a 4-neuron model for 2000 epochs with the AdamW optimiser, 0 weight decay, learning rate = 0.001. The resulting 31×31 Hessian has a minimum eigenvalue of -0.00217 , minimum absolute eigenvalue of 4.05×10^{-6} , and a maximum eigenvalue of 180.52. This indicates the landscape is highly ill-conditioned, and while the negative eigenvalue shows this is not a local minimum, AdamW struggles to converge. Furthermore, by finding the Frobenius norm of the off-diagonal terms relative to the norm of the full matrix, we find a coupling ratio of 0.66, which provides evidence weights are highly coupled.

We suggest the L-BFGS [23] optimiser instead. This is a limited-memory version of the BFGS optimisation algorithm, which uses an approximation of the inverse Hessian, built from previous parameter and gradient updates, to provide local curvature information. We theorise that this information will allow the optimiser to navigate the ill-conditioned landscape more effectively

and converge to a lower MSE minimum. For optimisation step k , define:

$$\begin{aligned} s_k &= x_{k+1} - x_k & y_k &= g_{k+1} - g_k \\ \rho_k &= \frac{1}{y_k^T s_k} \\ H_{k+1} &= (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \end{aligned}$$

where H_k is an approximation of the inverse Hessian. Unlike AdamW, which determines parameter update magnitudes independently based on previous gradients, L-BFGS transforms the joint gradient according to the local curvature. Indeed, models not only converge more quickly to a stable MSE and require fewer samples, but also achieve a substantially lower final MSE. In the earlier valley analogy, where AdamW can zig-zag between the steep cliffs of the valley, L-BFGS can use curvature information to follow the valley down to the minimum.

4 Activation Functions

An activation is a function applied to each neuron before output, to introduce non-linearity. Choice of activation function can dramatically impact the ability of a model to converge on an effective solution. It is even more significant in dynamical systems because the activation function directly contributes to the Jacobian of the vector field, which in turn influences the behaviour of the flow map. As in this paper we assume a 1-layer MLP, with weights W_1, W_2 , biases b_1, b_2 and activation function ϕ :

$$\begin{aligned} z &= W_1 x + b_1 \\ a &= \phi(z) \\ \hat{y} &= W_2 a + b_2 \end{aligned}$$

Hence, the model represents the vector field as:

$$f(x) = W_2 \phi(W_1 x + b_1) + b_2$$

The Jacobian of the vector field, $J_f(x)$, is:

$$\begin{aligned} J_f(x) &= \frac{\partial}{\partial x} (W_2 a + b_2) = W_2 \frac{\partial(\phi(z))}{\partial x} \\ &= W_2 \frac{\partial(\phi(z))}{\partial z} \frac{\partial z}{\partial x} \\ &= W_2 \text{diag} \left(\frac{\partial \phi}{\partial z_i} \right) W_1 \\ &= \sum_i \frac{\partial \phi}{\partial z_i} W_{2,:i} W_{1,i,:} \end{aligned}$$

Here, we see $J_f(x)$ is the sum of many rank-1 matrices, one contribution for each neuron.

4.1 ReLU

We first investigate ReLU, a conventional and widely-used activation function, and examine it's unsuitability for capturing the dynamics of the Lorenz system:

$$\text{ReLU}(z) = \begin{cases} 0, & \text{if } z < 0, \\ z, & \text{if } z \geq 0 \end{cases}$$

We notice that ReLU is piecewise linear. We will show that ReLU fails to capture the dynamics of the Lorenz equations. We trained models of width 32 using ReLU activation, for several random seeds, with L-BFGS until convergence, and took the best performing, with regards to Lyapunov spectrum. The Lyapunov spectrum initially looks passable, with:

- $\hat{\lambda}_1 = 0.288$
- $\hat{\lambda}_2 = -0.091$
- $\hat{\lambda}_3 = -14.15$

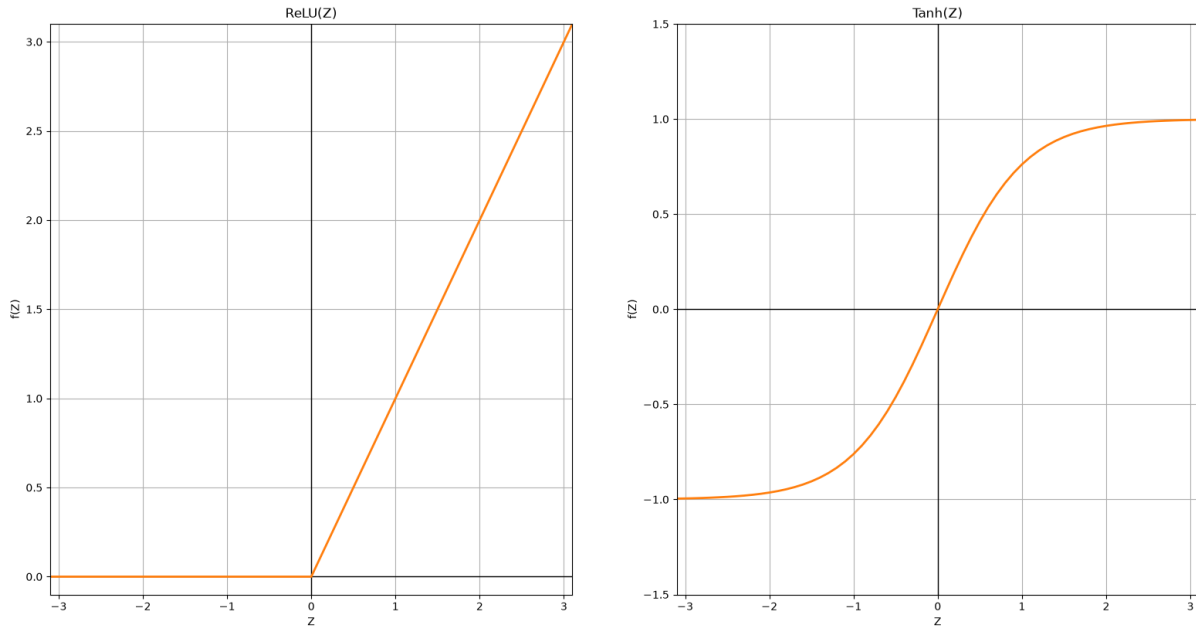


Figure 2: (Left) $\text{ReLU}(z)$ activation function (Right) $\text{Tanh}(z)$ activation function

This shows expansion in one direction, and contraction in another, just like the true L63.

However, the inspection of the SVD, in Figure 3, shows discontinuous behaviour incompatible with the true L63 geometry. The sinusoidal shape still vaguely appears, however, unlike the true system, amplitude doesn't appear to correlate with loop radius. There are very sharp drops in $\hat{\sigma}_3$, the contracting direction, to account for the fold areas.

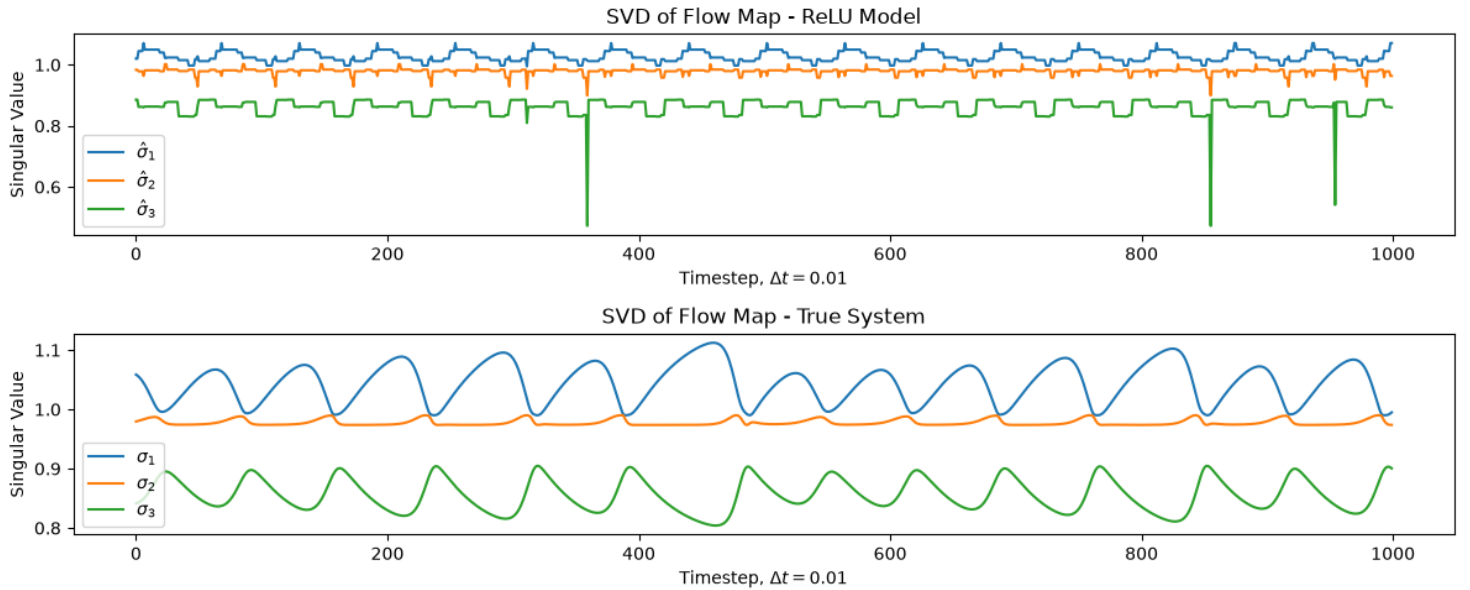


Figure 3: SVD for the ReLU Model (Top) and true Lorenz equations (Bottom)

This is to be expected, as the derivative of ReLU is:

$$\frac{d}{dz}(\text{ReLU}(z)) = \begin{cases} 0, & \text{if } z \leq 0, \\ 1, & \text{if } z > 0 \end{cases}$$

The derivative of neurons changes abruptly when a neuron passes from negative to positive. Hence, each neuron’s rank-1 contribution is either 0, when $z_i \leq 0$, or $W_{2,:,i}W_{1,i,:}$, when $z_i > 0$. This switches discontinuously over the trajectory. Consequently, the Jacobian of the vector field also varies discontinuously, changing only when a neuron changes sign. This means the singular values and vectors only ever change in discontinuous jumps. As a result, a ReLU network cannot represent the continuously varying rotations, contractions and expansions required by the L63 system.

In an arbitrarily wide network, this limitation can be overcome, as sufficiently many discontinuous changes can approximate the continuously varying flow map. However, this provides little practical relevance.

4.2 Tanh

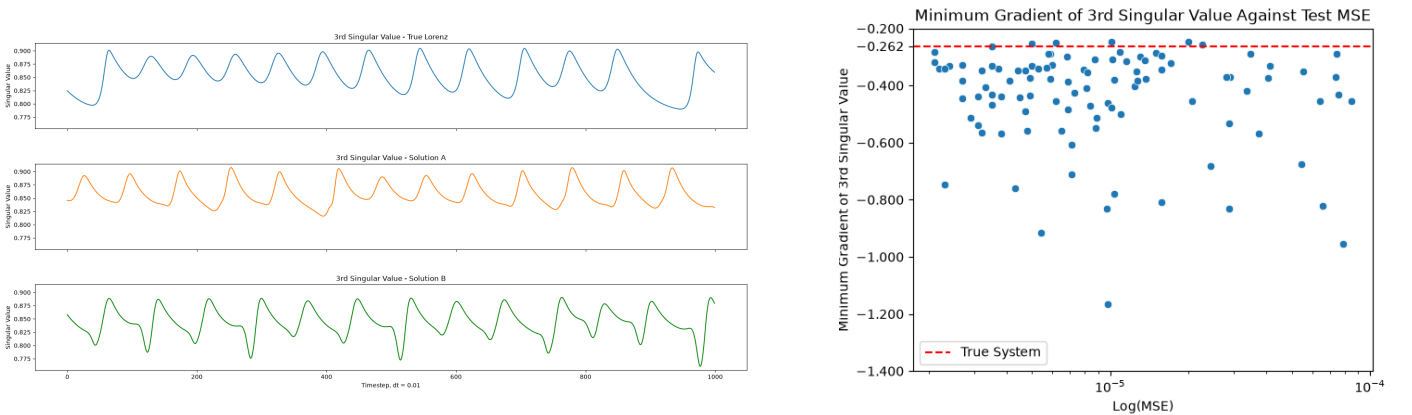
Next we will investigate tanh, the most common and widely assumed to be the best activation function for L63, shown in Figure 2.

Initially, we notice tanh is continuous and smooth, with a derivative which peaks at 0 and asymptotically approaches 0 as $x \rightarrow \pm\infty$. This ensures we will have a continuous change in Jacobian unlike with ReLU.

Indeed, SVD and Lyapunov spectra of tanh models proved more accurate than ReLU models. We trained 100 random initialisations for tanh models were trained to convergence, all with width 16. In this section, and the rest of this paper, confidence intervals are given with the 95% Wilson method [24]. The results showed 99% (95% C.I: 94.6–99.8%) of models had $\hat{\lambda}_1$, the first Lyapunov exponent, within 5% of the true λ_1 , and 77% (95% C.I: 67.8–84.2%) had $\hat{\lambda}_3$ within 5% of the true λ_3 . $\hat{\lambda}_3$ often proved harder for models to reproduce accurately. These results are promising.

However, inspection of the SVD showed evidence that models could be separated into 2 main solutions. Examples of these solutions are shown in Figure 4a. In one, the SVD showed smooth oscillations in amplitude that increase with the radius of the loop, as expected. However, many models showed $\hat{\sigma}_3$ decreasing slowly, before sharp drop in fold regions. This shows an under-contraction, followed by an overcorrection to allow the change of lobe required at these points. This often results in $\hat{\lambda}_3 < \lambda_3$.

We can imperfectly estimate this behaviour by finding the minimum gradient of $\hat{\sigma}_3$. The true L63 system has a minimum gradient of σ_3 of roughly -0.262 . 47% (37.5–56.7) of width-16 models have a minimum gradient of $\hat{\sigma}_3$ that differs from the true value by at least 50%. While this measure is imperfect, it indicates the problem well. Interestingly, Figure 4b shows that these solutions are not correlated with model MSE. This implies accurate local geometry is not necessarily correlated to the difference between the prediction and the true trajectory.



(a) Comparison of $\hat{\sigma}_3$ of the 2 categories of solution, with σ_3 , the true value. (Top) σ_3 , the true 3rd singular value (Middle) $\hat{\sigma}_3$ of a model showing smooth evolution, mirroring the true system. (Bottom) $\hat{\sigma}_3$ of a model showing sharp drops.

(b) Minimum Gradient of the 3rd singular value, $\hat{\sigma}_3$, of 100 trained models, with width 16 and a tanh activation.

Figure 4: Tanh activation analysis

This failure mode can be explained by the curvature of the activation function. The derivative of the $\tanh(z)$ is $\text{sech}^2(z)$. Each

*Although technically undefined, in practice we take $\left. \frac{d}{dz}(\text{ReLU}(z)) \right|_{z=0} = 0$, for computational convenience.

neuron contributes a rank-1 matrix to the Jacobian. We find that certain neurons become specialised for the strong contraction required in certain regions. As the magnitude of the neuron’s pre-activation changes quickly, $\text{sech}^2(z)$ also changes rapidly, as a result of the high curvature of the $\tanh(z)$ activation near $z = 0$. This causes the neuron’s contribution to the Jacobian, and consequently $\hat{\sigma}_3$, to sharply change, emphasised by the low number of contributing neurons. This results in a sharp drop in $\hat{\sigma}_3$ referring to an area of high contraction. Notably, these sharp drops happen in regions in which high contraction is required for lobe-swapping, suggesting the network has learnt to position the sharp contraction where it is most effective.

The other common solution systematically under-contracts the geometry, meaning the minima of $\hat{\sigma}_3$ is not as negative as the true system, however this does not seem to have an affect on the statistics of the attractor.

The observed solution seems determined via the initialisation of the weights. We investigated whether a tailored initialisation scheme could favour the smooth behaviour, but were unable to identify a scheme which improved consistency. Further investigation into initialisation may therefore be warranted.

We inspect the SVD of a particular model to demonstrate this failure mode. This model was selected from the 100 random seeds as it shows the sharp drops in $\hat{\sigma}_3$. We identify, by ablation, that the 13th neuron is specialised for contraction in the direction we are interested in. To isolate it’s effect, we fix $\text{sech}^2(z_{13})$ before the sharp drop, and compare the resulting $\hat{\sigma}_3$ to the original. We can see that the $\text{sech}^2(z_{13})$ drops from 0.86 to 0.26 quickly. This is seen in Figure 5, and shows that if this rapid change is prevented, $\hat{\sigma}_3$, doesn’t exhibit the sharp drop, providing direct evidence that the quick change in the neuron’s derivative is responsible for the overcorrection in contraction.

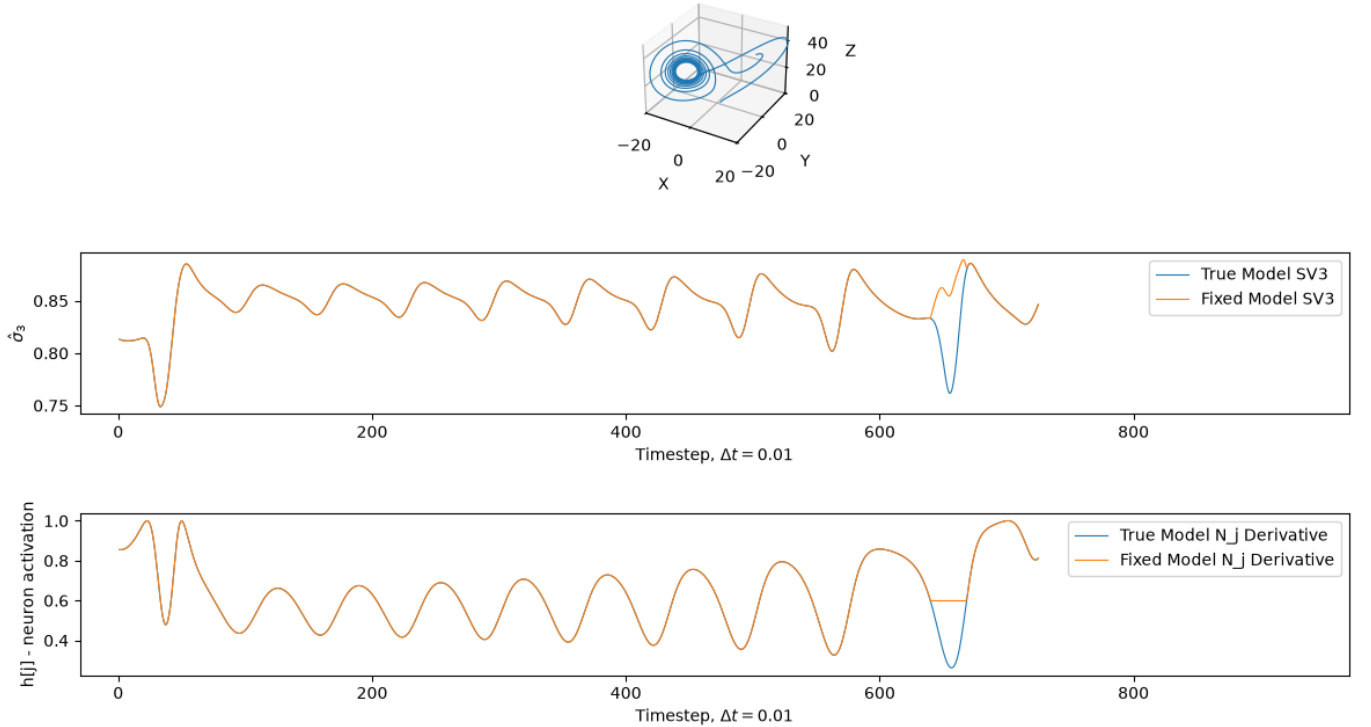


Figure 5: (Top) Autoregressive trajectory of the tanh model. (Middle) $\hat{\sigma}_3$ of the model, comparing fixed and true value. (Bottom) $\text{sech}^2(z_{13})$, derivative of a neuron specialised for contraction.

To confirm this hypothesis, we investigate another activation function, softplus, for which the gradient of the derivative is less steep. We predict the SVD of this function would be smoother and more representative of the true system.

4.3 Softplus

We will show that softplus is more consistent at capturing the local geometry of L63. Softplus is defined as:

$$\text{Softplus}(z) = \frac{1}{\beta} \ln(1 + e^{\beta z}),$$

and so the derivative is:

$$\frac{d}{dz}(\text{Softplus}(z)) = \frac{1}{1 + e^{-\beta z}}$$

Initially, we take $\beta = 1$.

Figure 6 compares the $\tanh(z)$ to $\text{softplus}(z)$, and $\frac{d}{dz}(\tanh(z))$ to $\frac{d}{dz}(\text{softplus}(z))$. We clearly see the gradient of $\frac{d}{dz}(\text{softplus}(z))$, with an absolute maximum gradient of 0.25, is less steep than $\text{sech}^2(z)$, which has a maximum absolute gradient of roughly 0.77.

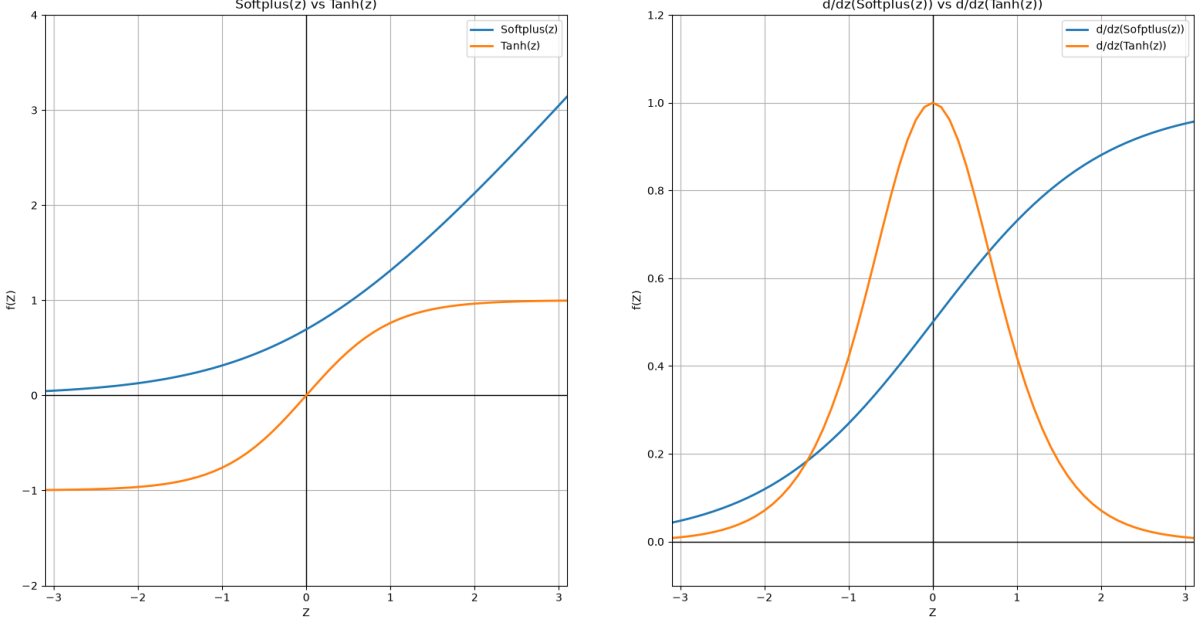


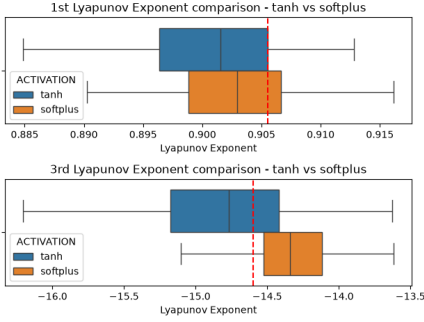
Figure 6: (Left) $\text{Softplus}(z)$ vs $\text{Tanh}(z)$. (Right) $\frac{d}{dz}(\text{Softplus}(z))$ vs $\frac{d}{dz}(\tanh(z))$

We similarly trained 100 models with the softplus activation, of width 16. 100% (95% C.I: 96.3–100%) of these models had $\hat{\lambda}_1$ within 5% of the λ_1 . 91% (95% C.I: 86.3–96.6%) of the model’s had $\hat{\lambda}_3$ within 5% of the true value, λ_3 . Most importantly, only 1% (95% C.I: 0.2–5.4%) of the softplus models had a minimum gradient of $\hat{\sigma}_3$, that differs by more than 50%, almost completely avoiding the failure mode of sharp corrections in contraction. Systematically, the softplus models would under-contract, shown by a minimum $\hat{\sigma}_3$ less than the true minimum value of σ_3 . Here, 98% (95% C.I: 93–99.4%) of models had a minimum less negative than the true system. This can be seen in Figure 7b. However, as mentioned earlier, this doesn’t seem to have an affect on attractor statistics. This shortcoming can be explained as to have high contraction at key fold points, we need the derivative of the neurons to change quickly, exactly the problem we try to avoid. It seems this is an trade-off problem, which show evidence of an inherent shortcoming of MLPs.

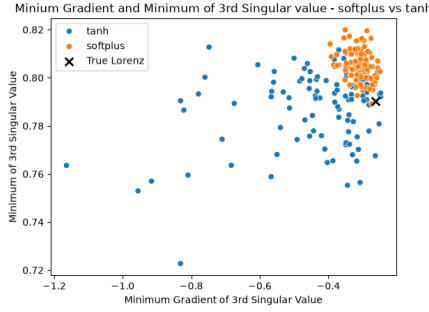
We can take this analysis one step further by investigating the effect of β on the failure mode of sharp drops in $\hat{\sigma}_3$. Recall:

$$\text{Softplus}(z) = \frac{1}{\beta} \ln(1 + e^{\beta z}).$$

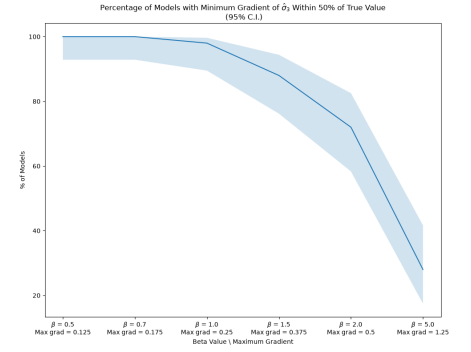
We can find that the maximum gradient of the derivative of softplus is $\frac{\beta}{4}$. By our theory, this implies that as β increases, more models would exhibit the sharp drops in $\hat{\sigma}_3$. Figure 7c shows the percentage of 50 models with varying β , which have a minimum gradient of $\hat{\sigma}_3$ that differs from the true value by at least 50%, imperfectly measuring this failure mode. We can see that it increases with β . We note, firstly, with $\beta < 0.5$ performance collapses, and the minima of the gradients of $\hat{\sigma}_3$ become unhelpful, as models fail to effectively capture the system at all. This is because the activations become more linear. We also note, this figure does not strictly imply that $\beta = 0.5$ is the best value to use, but only the best at avoiding of the failure mode of sharp drops in $\hat{\sigma}_3$. In fact, $\beta \in [0.5, 1]$ produce comparable results, when considering proportion of models with accurate Lyapunov spectra.



(a) (Top) $\hat{\sigma}_1$ distribution of tanh and softplus models. σ_1 represented by red dashed line. Outliers not shown. (Bottom) $\hat{\sigma}_3$ distribution of tanh and softplus models. σ_3 represented by red dashed line. Outliers not shown.



(b) Minimum gradients and minima of $\hat{\sigma}_3$ for softplus and tanh models, width



(c) The percentage of models with minimum gradient of $\hat{\sigma}_3$ that differs by more than 50% from the true value, showing a failure mode of sharp drops in $\hat{\sigma}_3$.

Figure 7: Tanh and Softplus Comparison

5 Width

The width of an MLP refers to how many neurons are in each hidden layer. It is intuitive that the wider the model, the more likely it is to be able to reproduce the long term dynamics and local geometry of the L63 equations. We trained 100 random initialisations of widths $\{4, 8, 16, 32\}$, and activations $\{\text{tanh}, \text{softplus}\}$, to evaluate this claim comparatively. The results agree with the hypothesis, with some interesting mechanistic findings. The table below shows the percentage of each width and each activation function to achieve metrics within a quantile bound of the true system. We look at them metrics of λ_1, λ_3 , and the minimum gradient of σ_3 .

MODEL	$\lambda_1 \pm 5\%$	$\lambda_1 \pm 2\%$	$\lambda_3 \pm 5\%$	$\lambda_3 \pm 2\%$	Min Grad $\sigma_3 \pm 50\%$	Min Grad $\sigma_3 \pm 20\%$
Softplus, 4	7% (3.5–13.9)	3% (1–8.5)	33% (24.8–43.1)	11% (6.3–18.8)	5% (2.2–11.3)	3% (1–8.5)
Softplus, 8	99% (94.6–99.8)	93% (86.3–96.6)	84% (75.6–89.9)	49% (39.4–58.7)	92% (85–95.9)	61% (51.2–70)
Softplus, 16	100% (96.3–100)	97% (91.5–99)	91% (86.3–96.6)	52% (42.3–61.5)	99% (94.6–99.8)	62% (52.2–370.9)
Softplus, 32	100% (96.3–100)	93% (86.3–96.6)	96% (91.5–99)	62% (52.2–70.9)	99% (94.6–99.8)	79% (70–85.8)
Tanh, 4	2% (0.6–7)	2% (0.6–7)	10% (5.5–17.4)	4% (1.6–9.8)	12% (7–19.8)	1% (0.2–5.4)
Tanh, 8	85% (76.7–90.7)	72% (62.5–79.9)	35% (26.4–44.7)	14% (8.5–22.1)	67% (57.3–75.4)	33% (24.6–42.7)
Tanh, 16	99% (94.6–99.8)	89% (81.4–93.7)	77% (67.8–84.2)	38% (29.1–47.8)	53% (43.3–62.5)	19% (12.5–27.8)
Tanh, 32	99% (94.6–99.8)	99% (94.6–99.8)	85% (76.7–90.7)	34% (25.5–43.7)	54% (44.3–63.4)	27% (19.3–36.4)

Table 1: Quantiles of error for each architecture

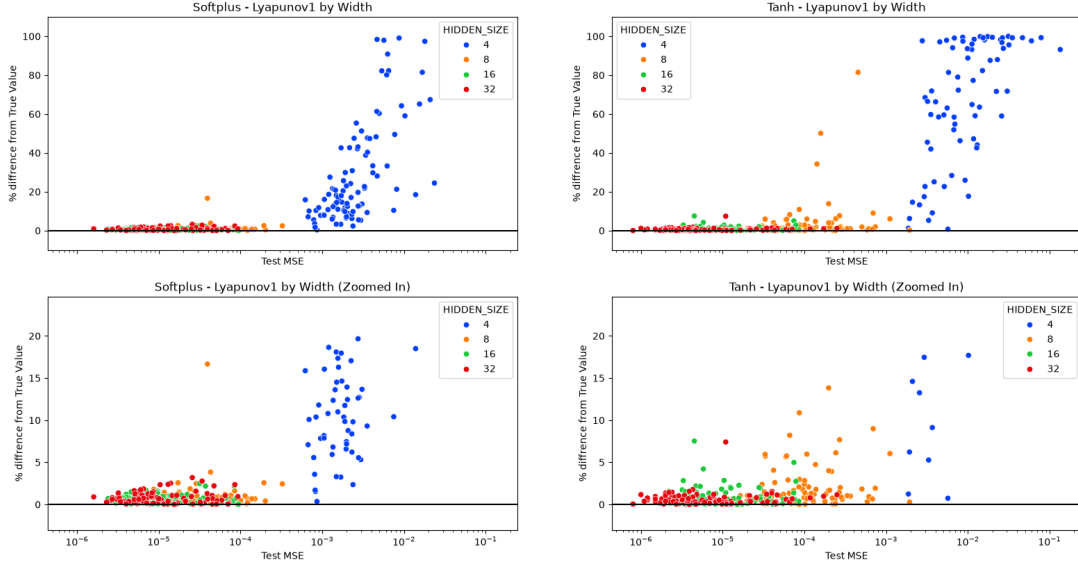
Before plotting, we must remove extreme values, which did not converge to a chaotic system. This only occurs in 1% of width-4 softplus models and 27% of width-4 tanh models. These are removed from future analysis.

By inspecting Figure 8, we can clearly see that in both Lyapunov exponents, width-4 models have considerably greater variation than wider models, and are less likely to preserve the correct long-term dynamics. In tanh models, there seems a clearer dependence on width: in softplus models, particularly for $\hat{\lambda}_1$, models of widths 8, 16 and 32 are of comparable consistency. In tanh models, we see that consistency increases with width. However, in all cases, the best performing models of each width are of comparable accuracy. It is therefore the consistency of finding a good solution that is influenced by width, rather than the quality of best solution.

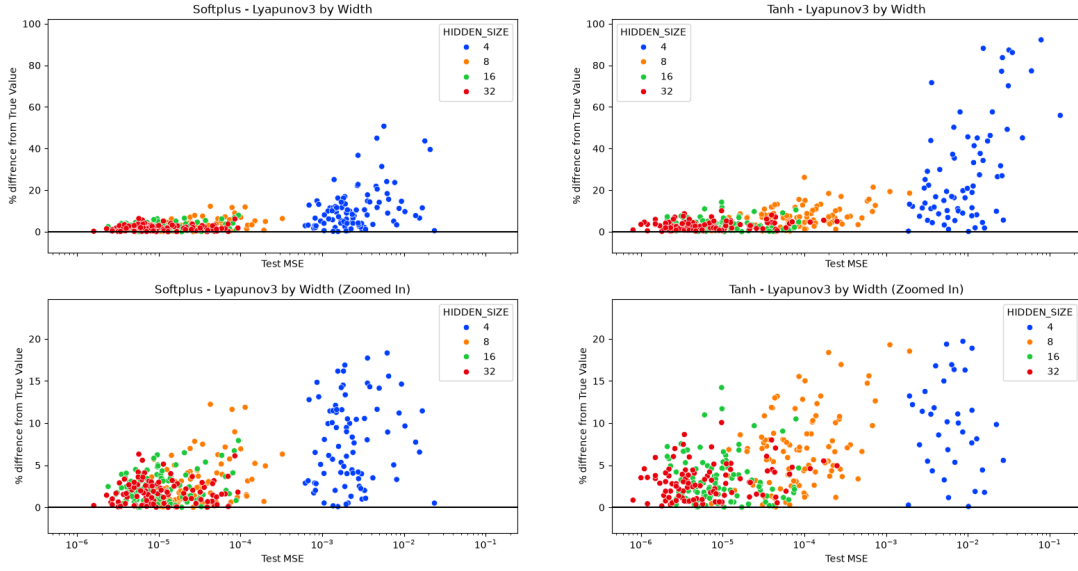
There is important differences between $\hat{\lambda}_1$ and $\hat{\lambda}_3$. We can see the latter is more variable across seeds, suggesting it is more difficult to reproduce.

These plots again back up the conclusion that softplus more consistently able to capture the dynamics of the L63 equations than tanh.

We next investigate the common solutions capturing in the "Activation" section, namely sharp drops in $\hat{\sigma}_3$ represented by a small minimum gradient of $\hat{\sigma}_3$, and a systematic under-contraction represented by a minimum $\hat{\sigma}_3$ higher than the true system. This can be seen in Figure 9, and remove extreme values for plotting only, namely where the minimum of $\hat{\sigma}_3 < 0.5$. These values were only provided by width-4 models, with 2 models removed with softplus activation and 11 removed from tanh.



(a) (Top Left) Percentage error of $\hat{\lambda}_1$ from the true value, for softplus models across widths. (Bottom Left) Percentage error of $\hat{\lambda}_1$ from the true value, for softplus models across widths. (zoomed in at $< 20\%$). (Top Right) Percentage error of $\hat{\lambda}_1$ from the true value, for tanh models across widths. (Bottom Right) Percentage error of $\hat{\lambda}_1$ from the true value, for tanh models across widths. (zoomed in at $< 20\%$).



(b) (Top Left) Percentage error of $\hat{\lambda}_3$ from the true value, for softplus models across widths. (Bottom Left) Percentage error of $\hat{\lambda}_3$ from the true value, for softplus models across widths. (zoomed in at $< 20\%$). (Top Right) Percentage error of $\hat{\lambda}_3$ from the true value, for tanh models across widths. (Bottom Right) Percentage error of $\hat{\lambda}_3$ from the true value, for tanh models across widths. (zoomed in at $< 20\%$).

Figure 8: Lyapunov Spectra Analysis on Widths

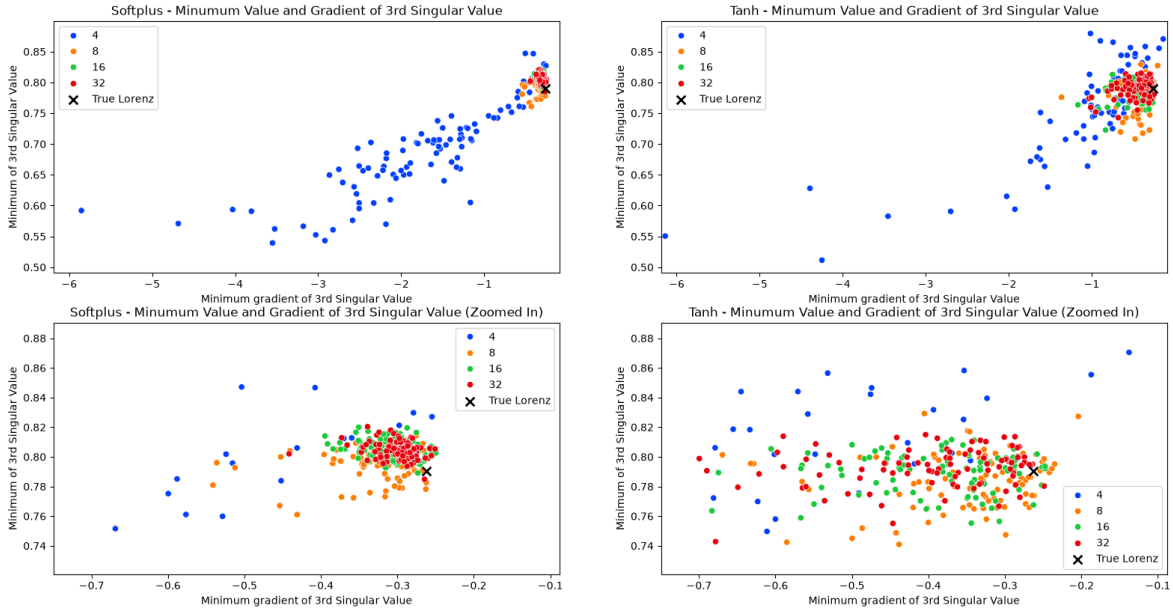


Figure 9: (Top Left) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths. (Bottom Left) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths (zoomed in). (Top Right) Minimum gradient and minimum of $\hat{\lambda}_3$ for tanh models across widths. (Bottom Right) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths (zoomed in).

We, as usual, notice a substantial difference between width-4 models and the larger models, with width 4 having much more variability and, in general, lower accuracy. Unlike the Lyapunov spectra, the best performing width-4 models are noticeably worse than almost all larger models. For width-4 models, the minimum gradient of $\hat{\sigma}_3$ is almost always less than the true value. This is the same failure mode as diagnosed earlier with the tanh model. Whereas before we had a sharp drop in a particular neuron, which was specialised for contraction, this sharp drop can be explained by the overall drop in the derivative of neurons. With fewer neurons, each rank-1 contribution is more influential on the overall Jacobian. At fold points, even a small drop in the derivative of neurons provides a big impact on the Jacobian, resulting in a sharp change in the singular values, and in particular a sharp drop in $\hat{\sigma}_3$.

In the zoomed in plots, we can see that the softplus models has much less variability than the tanh models, and in general are closer to the true values. Indeed, as seen earlier, for each width, the tanh models have a lot more models in which the minimum gradient of $\hat{\sigma}_3$ is much lower than the true system. That being said, the best performing model of each are of similar accuracy.

We notice, in the softplus model, the minimum of $\hat{\sigma}_3$ is almost always higher than the true system. This was touched on earlier, and represents a systematic under-contraction of the model, however it is not clear that it has any effect on the statistics of the attractor. This shortcoming can be attributed to the trade-off between contraction capability and alleviation of sharp drops in $\hat{\sigma}_3$, which seems to be an inherent problem of small MLP's.

The minimum gradient of $\hat{\sigma}_3$ of tanh models does not seem to improve with width. In fact, from Table 1, 33% (95% C.I: 24.6-42.7%) of the width-8 models are within 20% error of the true minimum gradient σ_3 , as oppose to 19% (95% C.I: 12.5-27.8%) and 27% (95% C.I: 19.3-36.4%) of width-16 and width-32 models respectively. Unlike softplus, it can be seen that the tanh models are roughly split between below and above the minimum gradient of σ_3 . As mentioned earlier, these refer to the 2 possible categories that a model tends to show in the SVD decomposition.

6 Generalisation: Lorenz-96

To test the generalisation of these findings, we investigate the Lorenz-96 (L96) equations. [25] The L96 equations were introduced by Edward Lorenz in his 1995 seminar titled "Predictability - A Problem Partly Solved", and later written in a paper of the same name. As with L63, they are an idealised model for dynamics in the atmosphere:

$$\begin{aligned}\dot{X}_k &= (X_{k+1} - X_{k-2})X_{k-1} - X_k + F, \\ X_{k+N} &= X_k\end{aligned}$$

for $k \in [1, N]$, and where F is a constant. This represents N equally-spaced spatial domains. For our purpose, we set $N = 8, F = 8$.

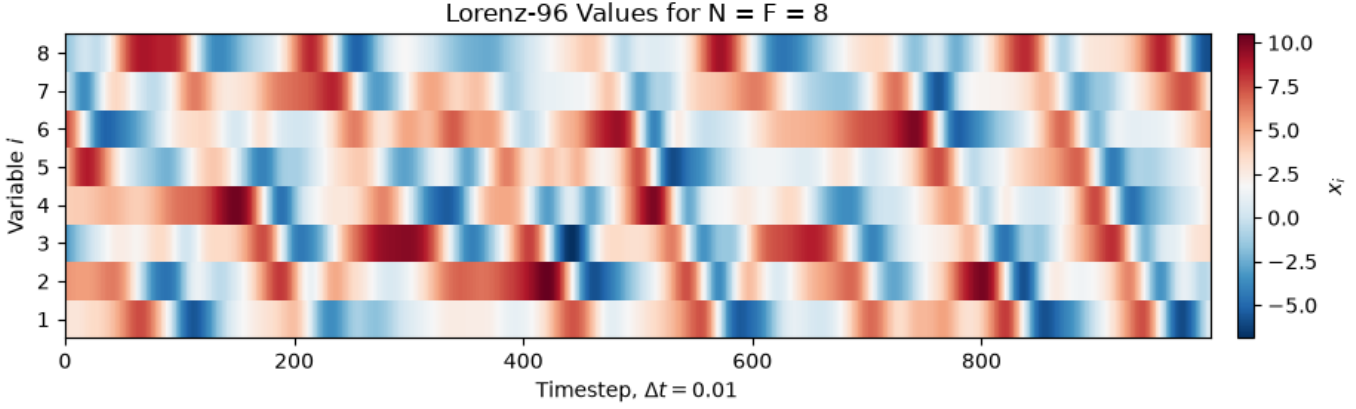


Figure 10: Heat map of X_i over a trajectory of 1000 time steps for the Lorenz-96 equations.

λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7	λ_8
1.6435	0.3932	0.0008	-0.4232	-0.9879	-1.5237	-2.4940	-4.6089

Table 2: Lyapunov Spectrum of L96

We trained random initialisations of activations $\{\tanh, \text{softplus}(\beta = 1)\}$ and width $\{24, 32, 64\}$. Due to the increased complexity of the system, we find width 24 to be the minimum reasonable width to analyse, as smaller widths result in values exponentially increasing when integrated auto-regressively. We chose to only train 50 random seeds of each mode. For succinctness, we focus on the strongest expansive and contractive directions.

We firstly notice that although softplus models of width 24 can very effectively capture the dynamics of the system, in both long-term and instantaneous expansion and contraction, the same is not true for tanh models of the same width. Of the 50 trained seeds of width 24 and activation tanh, only 24% (95% C.I: 14.3–37.4%) had $\hat{\lambda}_1$ and $\hat{\lambda}_8$ within 10% of the true values, as oppose to 98% (95% C.I: 89.5–99.6%) of softplus models. In fact, only 76% (95% C.I: 62.6–85.7%) were chaotic. This effectively justifies that for respectively small models, softplus is more consistent at capturing the necessary dynamics of the system than tanh.

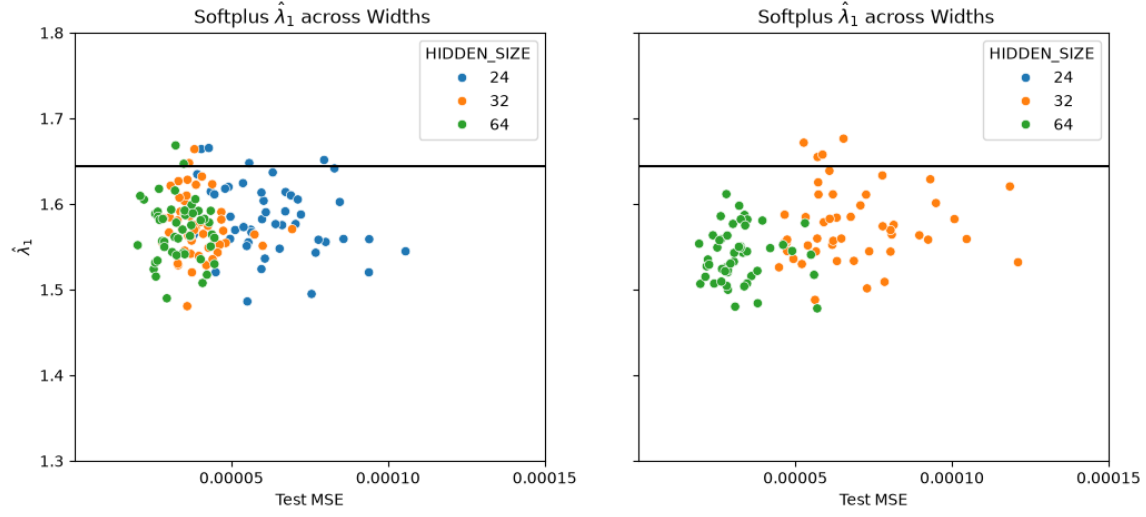
On inspection of Figure 11, the Lyapunov spectrum of softplus models of all widths are equally consistent, including the mean absolute error (MAE), which shows width-24 models are just as effective at consistently capturing the dynamics of the system as width-64 models. This is surprising given our earlier results on L63.

Interestingly, it seems that width-32 tanh models are more effective at capturing the long-term dynamics of the system than width-64 tanh models, most obvious in Figure 11c, in which width-32 models have a consistently lower MAE than width-64. Indeed, 92% (95% C.I: 81.2–96.8%) of width-32 models have an MAE less than 0.07, whereas only 6% (95% C.I: 2.1–16.2%) of width-64 models do. This is the opposite of the result from the L63 experiment.

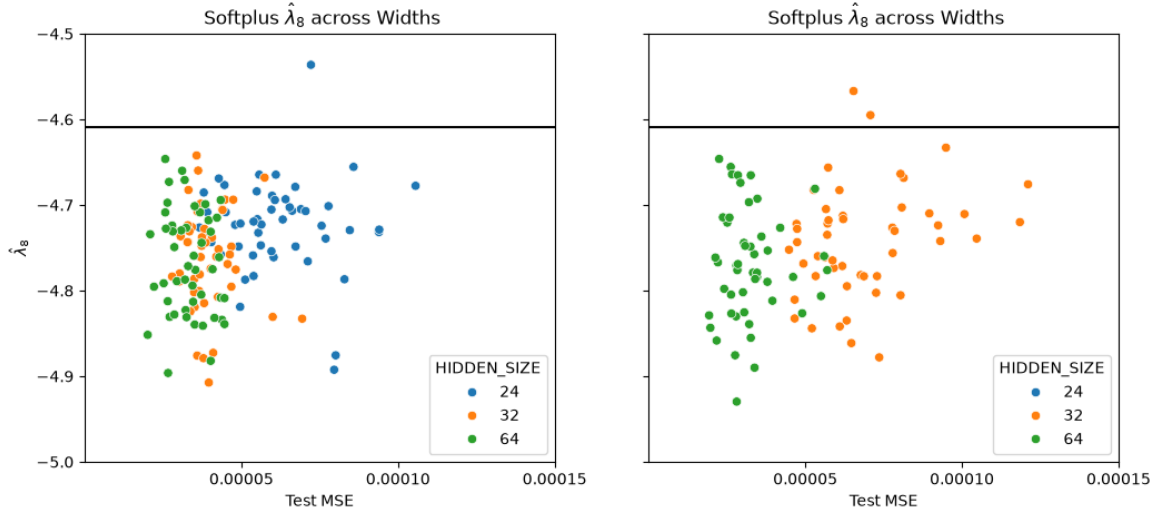
Together, these findings give evidence that there is a minimum viable width needed to preserve the complex dynamics of the system. Models with width lower this point fail to consistently represent the system it is trained on. Further increasing the width seems to have little additional benefit, and in some cases may adversely affect the ability of the model to recreate the system. We theorise there is a minimum viable width is dependant on the activation function used. For a model of suitable width to have low 1-step error, it must represent the underlying dynamics sufficiently accurately. To increase consistency and accuracy further, more depth would need to be added.

We also confirm that at width-64, softplus is better than tanh at preserving the Lyapunov spectrum of the true system, with 90% (95% C.I: 78.6–95.7%) of models having MAE less than 0.07, however at width-32, both activations perform equally.

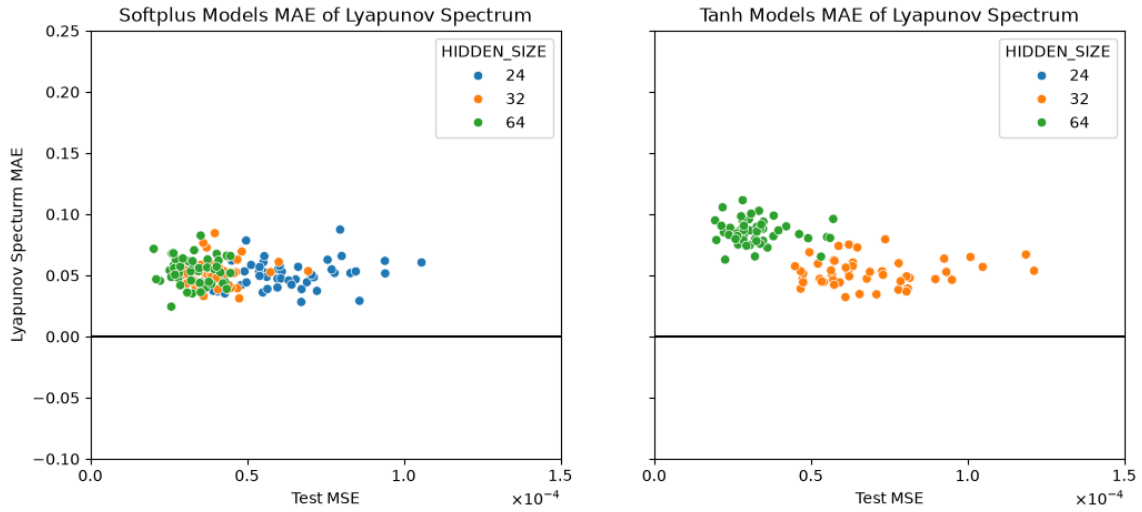
On inspection of Figure 12, we can see that in softplus models, the width-24 models are more variable than the wider models, as we concluded from L63. However, the effect is much lower. Although much less extreme than in the case of L-63, tanh models have a larger spread in minimum gradient of $\hat{\sigma}_8$. This can be attributed to the higher curvature of the activation function. The



(a) (Left) $\hat{\lambda}_1$ of trained models across width, for softplus models. (Right) $\hat{\lambda}_1$ of trained models across width, for tanh models.



(b) (Left) $\hat{\lambda}_8$ of trained models across width, for softplus models. (Right) $\hat{\lambda}_8$ of trained models across width, for tanh models.



(c) (Left) Mean absolute error of the Lyapunov spectra of the model compared to the true system, for softplus models. (Right) Mean absolute error of the Lyapunov spectra of the model compared to the true system, for tanh models.

Figure 11: Lyapunov Spectra Analysis on Widths (L96)

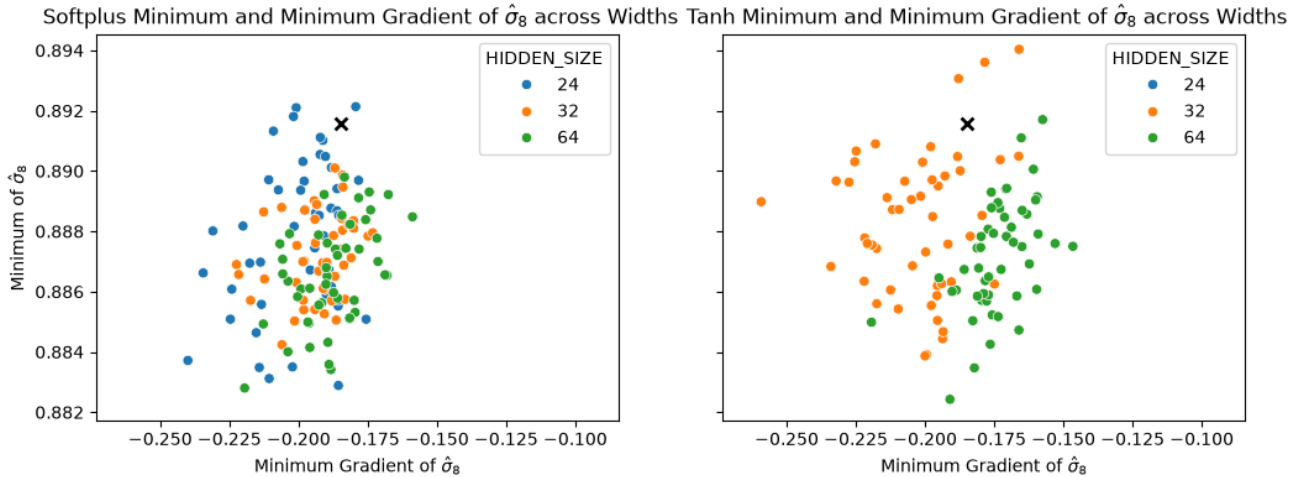


Figure 12: (Left) Minimum and Minimum Gradient of $\hat{\sigma}_8$ for Softplus Models across Width (Right) Minimum and Minimum Gradient of $\hat{\sigma}_8$ for tanh Models across Width

reduced effect of this, on the L96, can be speculated as the larger hidden size, as each neuron contributes less to the overall Jacobian. Interestingly, all modes of models seem to have a tendency to have a minimum of $\hat{\sigma}_8$ less than the true system.

7 Conclusion

This project investigates the ability and inability of simple, minimally structured neural networks for capturing the long-term dynamics and instantaneous geometry of chaotic systems. Using Lorenz-63 as a primary test-bed, we comparatively evaluated various training and architectural choices, using Lyapunov spectra and local singular value decomposition of the Jacobian. These metrics revealed several shortcomings of common decisions in successfully learning a chaotic dynamical system.

The choice of optimiser is vital in small networks. We found that the loss landscape was strongly ill-conditioned and parameters were highly coupled. Consequently, AdamW, which calculates parameter updates largely independently, fails to effectively converge to a low-MSE minimum. On the other hand, L-BFGS is much more effective, and efficient, for this problem, as it utilises an estimate for the local inverse Hessian, and can therefore more effectively navigate the ill-conditioned landscape and converge on a more accurate minimum. This suggests, more generally, that curvature-aware optimisers are more suited for small, highly coupled neural networks, such as those built for chaotic dynamical systems.

We found choice of activation function to be key in the ability of a simple model to be able to capture the dynamics of the chaotic system. We first formalised the argument that ReLU, a piecewise activation function, is unable to reproduce the smooth evolution of local geometry of Lorenz-63. This arises from the discrete changes in the model’s Jacobian, caused by the piecewise-constant derivative of the ReLU function. Therefore, each neuron contributes either 0, or a constant rank-1 matrix, to the Jacobian, resulting in a Jacobian that can only change when neurons switch activation state.

We investigated tanh, a commonly used activation function for dynamical systems. Although both the attractor and Lyapunov spectrum results were much more accurate, singular-value decomposition showed two distinct classes of solution. Some models effectively reproduced the smooth evolution in contraction, however others showed a sharp correction in contraction. This result was shown to be caused by the rapid change in the value of the derivative of the activation function, due to a high curvature.

We suggested the activation function of softplus, which overcame this failure mode, and substantially reduced the variability of long term dynamics and local geometry in small models. This result generalised to a more complex dynamical system. This can be explained by the lower curvature of softplus. However, softplus systematically under-contracts in regions of strong contraction. This emphasises an important trade-off of MLP’s. In reducing the curvature of the activation function, we also limit the ability of the models to produce areas of strong contraction.

Network width revealed another important feature of reproducing the dynamics of the attractor. In the simple Lorenz-63 study, increasing width would increase the consistency of models to reproduce the long and short term dynamics of a system, but not necessarily improve the accuracy of the best model. However, when investigating Lorenz-96, we found that increasing width did not monotonically improve the consistency of accurate models, but instead that there was a minimum viable width, for which higher widths would have no additional benefit, or in some cases, reduce accuracy of dynamics.

Overall, these results show that a small 1-step error is not sufficient to establish whether a neural network has learned the long-term dynamics and instantaneous geometry of a dynamical system. Models with similar MSE can have substantially different local geometries and Lyapunov spectra. These measures therefore provide complimentary and necessary information of the success of a model in recreating a chaotic dynamical system.

These results effectively demonstrate the complex affects of training regime and architectural decisions for a networks ability to capture a dynamical system. Simple MLP's can reproduce important features of a chaotic system, but their success cannot be guaranteed by a low loss. The variability of the dynamics of trained models, together with the activation function trade-off, highlight limitations of simple MLPs, and provide insight as to why MLPs are not suitable as general-purpose models for complex dynamical systems.

8 Future Work

Future work on this topic would include comparative study on models of different architectures, namely RNN's, Neural ODE's and reservoir computing. These architectures relax some structural assumptions of the MLP.

Furthermore, dynamical systems often have various 'chaotic modes' based on parameters and initial conditions. In the case of Lorenz-63, this can be controlled by the parameters (β, σ, ρ) . There is potential for interesting findings when we consider the minimum viable network required to consistently capture the dynamics across these regimes i.e. does a non-chaotic regime of Lorenz-63 require a smaller network to consistently capture the dynamical properties? This question could be further investigated with the double pendulum system, in which regime is determined by initial energy of the system.

APPENDIX

Derivation of the Tangent Propagator

We have an unperturbed point, $\bar{x}$, and perturb it with the perturbation x' , so the perturbed point, x is:

$$x(t) = \bar{x}(t) + x'(t)$$

First, for sufficiently small perturbations, dynamics may be linearised. Thus, we have $\dot{x}'(t) = J(t)x'(t)$. Here, $J(t) = F'(x)|_{x=x(t)}$. The tangent propagator, $M(t, t_0)$, is the map which takes a the perturbation., x' , at time t_0 to the perturbation at time t :

$$x'(t) = M(t, t_0)x'(t_0)$$

As choice of initial time is arbitrary, we set $t_0 = 0$. We have:

$$x'(t) = M(t, 0)x'(0)$$

$$\dot{x}' = \frac{dM}{dt}x'(0)$$

$$J(t)x'(t) = \frac{dM}{dt}x'(0)$$

$$\frac{dM}{dt} = J(t)M$$

Then, by integrating:

$$\int_0^t \frac{dM(s, 0)}{ds} ds = \int_0^t J(s)M(s, 0) ds$$

In general, the multiplication of Jacobians does not commute:

$$J(t_1)J(t_2) \neq J(t_2)J(t_1)$$

Hence, we must preserve the order of multiplication.

$$M(t, 0) - M(0, 0) = \int_0^t J(s)M(s, 0) ds$$

. Notice $M(0,0) = I$. We substitute:

$$M(t, 0) = I + \int_0^t J(s_1) \left[I + \int_0^{s_1} J(s_2) M(s_2, 0) ds_2 \right] ds_1$$

$$M(t, 0) = I + \int_0^t J(s_1) ds_1 + \int_0^t \int_0^{s_1} J(s_1) J(s_2) M(s_2, 0) ds_2 ds_1$$

Repeating this substitution, we obtain the Dyson Series, which can be written compactly:

$$M(t, 0) = \mathcal{T} \exp \left(\int_0^t J(s) ds \right),$$

where $\mathcal{T}$ is the time-ordering operator. Recall, the first Lyapunov Exponent, λ_1 , is the long-term averaged exponential rate of growth of a perturbation:

$$\|x'(t)\| \sim e^{\lambda_1 t} \|x'(0)\|$$

Rearranging, and finding the limit:

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|x'(t)\|}{\|x'(0)\|}$$

Hence, we have

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|M(t, 0)x'(0)\|}{\|x'(0)\|}$$

References

- [1] A. M. Lyapunov. *The General Problem of the Stability of Motion*. 1892.
- [2] G. H. Golub and C. Reinsch. “Singular value decomposition and least squares solution”. In: *Numerische Mathematik* 14.5 (1970), pp. 403–420. DOI: 10.1007/bf02163027.
- [3] E. N. Lorenz. “Deterministic Nonperiodic Flow”. In: *Journal of the Atmospheric Sciences* 20.2 (1963), pp. 130–141. DOI: 10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2.
- [4] K. T. Alligood, T. D. Sauer, and J. A. Yorke. *CHAOS: An Introduction to Dynamical Systems*. 2000.
- [5] S. H. Strogatz. *Nonlinear Dynamics and Chaos*. 2018.
- [6] P. R. Vlachas et al. “Data-driven forecasting of high-dimensional chaotic systems with long short-term memory networks”. In: *Proc. R. Soc. A* 474.2213 (2018). DOI: 10.1098/rspa.2017.0844.
- [7] S. Hochreiter and J. Schmidhuber. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [8] J. L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211. DOI: 10.1016/0364-0213(90)90002-E.
- [9] S. Shahi, F. H. Fenton, and E. M. Cherry. “Prediction of chaotic time series using recurrent neural networks and reservoir computing techniques: A comparative study”. In: *Machine Learning with Applications* 8 (2022). DOI: 10.1016/j.mlwa.2022.100300.
- [10] H. Jaeger. *The “Echo State” Approach to Analysing and Training Recurrent Neural Networks*. Tech. rep. GMD-Report 148. German National Research Center for Information Technology, 2001.
- [11] W. Maass, T. Natschlager, and H. Markram. “Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations”. In: *Neural Computation* 14.11 (2002), pp. 2531–2560. DOI: 10.1162/089976602760407955.
- [12] D. J. Gauthier et al. “Next generation reservoir computing”. In: *Nat Commun* 12.5564 (2021). DOI: 10.1038/s41467-021-25801-2.
- [13] J. Pathak et al. “Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach”. In: *Phys. Rev. Lett.* 120.2 (2018). DOI: 10.1103/PhysRevLett.120.024102.
- [14] G. Margazoglou and L. Magri. “Stability analysis of chaotic systems from data”. In: *Nonlinear Dyn* 111 (2023), pp. 8799–8819. DOI: 10.1007/s11071-023-08285-1.
- [15] J. Park, N. Yang, and N. Chandamoorthy. “When are dynamical systems learned from time series data statistically accurate?” In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 43975–44008.
- [16] R. T. Q. Chen et al. “Neural Ordinary Differential Equations”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.

- [17] A. Ribes B. Balogh D.Saint-Martin. “A Toy Model to Investigate Stability of AI-Based Dynamical Systems”. In: *Geophysical Research Letters* 48.8 (2021). DOI: 10.1029/2020GL092133.
- [18] Z. Li and S. Ravela. “Neural Networks as Geometric Chaotic Maps”. In: *IEEE Transactions on Neural Networks and Learning Systems* 34.1 (2023), pp. 527–533. DOI: 10.1109/TNNLS.2021.3087497.
- [19] G.Benettin et al. “Lyapunov Characteristic Exponents for smooth dynamical systems and for hamiltonian systems; a method for computing all of them. Part 1: Theory”. In: *Meccanica* 15 (1980), pp. 9–20. DOI: 10.1007/BF02128236.
- [20] A. S. Householder. “Unitary Triangularization of a Nonsymmetric Matrix”. In: *J. ACM* 5.4 (1958), pp. 339–342. DOI: 10.1145/320941.320947.
- [21] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. 2008.
- [22] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. 2019.
- [23] D. C. Liu and J. Nocedal. “On the limited memory BFGS method for large scale optimization”. In: *Mathematical Programming* 45 (1989), pp. 503–528. DOI: 10.1007/BF01589116.
- [24] E. B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference”. In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212. DOI: 10.1080/01621459.1927.10502953.
- [25] E. N. Lorenz. “Predictability: A Problem Partly Solved”. In: *Seminar on Predictability*. Vol. 1. 1996, pp. 1–18.